🚀 NIVIX PROTOCOL - MASTER DOCUMENTATION

Version: 2.0
Last Updated: January 1, 2026
Status: Development/Testing Phase (Solana Devnet)
Company: NivixPe Private Limited
Incubated at: Bennett Hatchery, Bennett University

📋 TABLE OF CONTENTS

1. Executive Summary
2. Project Overview
3. System Architecture
4. Technology Stack
5. Project Structure
6. Blockchain Integration
7. Payment Gateway Integration
8. KYC/Compliance System
9. On-Ramp System
10. Off-Ramp System
11. Liquidity Pools
12. Treasury Management
13. API Documentation
14. Deployment Guide
15. Commands Reference
16. Security & Key Management
17. Legal & Regulatory Compliance
18. Production Readiness Checklist
19. Troubleshooting
20. Project Status & Roadmap

---

1. EXECUTIVE SUMMARY

1.1 What is Nivix Protocol?

Nivix Protocol is a production-ready, dual-blockchain payment platform that enables fast, secure, and compliant international money transfers. The system integrates:

- **Solana Blockchain (Devnet)**: High-performance payment processing with SPL tokens
- **Hyperledger Fabric**: Private KYC/AML compliance ledger
- **Razorpay Payment Gateway**: Fiat on-ramp integration (Test Mode)
- **RazorpayX Payouts**: Fiat off-ramp for bank transfers
- **React Frontend**: Modern user interface with wallet integration
- **Node.js Bridge Service**: Cross-chain orchestration and API gateway

1.2 Key Metrics

Metric	Value
Transaction Speed	< 1 second (Solana)
Transaction Cost	< $0.001
API Response Time	< 200ms average
On-Ramp Success Rate	100%

Supported Currencies	USD, INR, EUR, GBP, JPY, CAD, AUD
Liquidity Pools	20+ deployed

1.3 Current Status

Component	Status	Completion
On-Ramp (Fiat → Crypto)	✅ Fully Operational	100%
KYC/Compliance	✅ Complete	80%
Off-Ramp (Crypto → Fiat)	⏳ Ready for Testing	70%
Multi-Currency Pools	⏳ In Progress	60%
Frontend	⚠️ Needs Work	35%
Treasury System	❌ Early Stage	25%

---

2. PROJECT OVERVIEW

2.1 Problem Statement

Traditional cross-border remittance services suffer from:

- **High Fees**: 5-10% per transaction
- **Slow Settlement**: 3-5 business days
- **Limited Accessibility**: Restricted banking hours and infrastructure

2.2 Nivix Solution

Blockchain-based payment infrastructure providing:

- **Low Fees**: < 0.5% platform fee
- **Fast Settlement**: < 2 minutes end-to-end
- **24/7 Availability**: Blockchain never sleeps
- **Compliance**: Integrated KYC/AML

2.3 Business Model

1. **Transaction Fees**: 0.5-1% markup on currency conversion
2. **Exchange Rate Spread**: Minimal spread on conversions
3. **Service Fees**: Per-transaction processing fees

2.4 User Flow

```
1. User Registration → KYC Verification (Hyperledger Fabric)
2. Deposit INR → Razorpay Payment Gateway
3. Receive Tokens → Solana Blockchain (SPL Tokens)
4. Transfer Tokens → Cross-border via Solana network
5. Convert to Fiat → RazorpayX Payout to recipient bank account
```

---

3. SYSTEM ARCHITECTURE

3.1 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                        NIVIX PROTOCOL                        │
└──────────────────────────────────────────────────────────────┘


┌────────────────┐          ┌────────────────┐          ┌────────────────┐
│   Frontend     │◄────────►│ Bridge Service │◄────────►│  Blockchain    │
│   (React)      │  HTTPS   │   (Node.js)    │   RPC    │   Layers       │
│   Port: 3000   │          │   Port: 3002   │          │                │
└────────────────┘          └────────────────┘          └────────────────┘
                                    │                        │
                                    │                        ├─► Solana
                                    │                        │   (Payments)
                                    │                        │
                                    ├──────────────┐         └─► Hyperledger
                                    │              │             (KYC/AML)
                            ┌───────▼────┐    ┌────▼──────────┐
                            │ Razorpay   │    │  Treasury     │
                            │  Gateway   │    │  Manager      │
                            └────────────┘    └───────────────┘
```

3.2 Component Interaction

1. **User Interface** → Frontend (React + Material-UI)
2. **API Gateway** → Bridge Service (Express.js)
3. **Payment Processing** → Solana Blockchain (Anchor/Rust)
4. **Compliance** → Hyperledger Fabric (JavaScript/Go Chaincode)
5. **Fiat Integration** → Razorpay/RazorpayX

3.3 Data Flow

1. **User Registration**: KYC data → Hyperledger Fabric (private)
2. **Payment Initiation**: INR deposit → Razorpay → Bridge Service
3. **Token Minting**: Bridge Service → Solana blockchain → User wallet
4. **Transfer**: User wallet → Solana blockchain → Recipient wallet
5. **Off-Ramp**: Token burn → RazorpayX → Bank account

---

4. TECHNOLOGY STACK

4.1 Frontend

Technology	Version	Purpose
React	18.2.0	UI Framework
TypeScript	4.9.5	Type Safety
Material-UI	5.15.0	Component Library
Solana Wallet Adapter	0.15.38	Wallet Integration
React Router DOM	6.14.2	Routing

4.2 Backend (Bridge Service)

Technology	Version	Purpose
Node.js	18+	Runtime

Express.js	4.21.2	Web Framework
@solana/web3.js	1.98.4	Solana Integration
@solana/spl-token	0.4.13	Token Operations
fabric-network	2.2.18	Hyperledger SDK
axios	1.11.0	HTTP Client

4.3 Blockchain

Technology	Version	Purpose
Solana	Devnet	Payment Settlement
Anchor Framework	0.31.1	Smart Contract Framework
Rust	2021 Edition	Smart Contract Language
Hyperledger Fabric	2.2.18	Compliance Ledger

4.4 Payment Gateway

Service	Mode	Purpose
Razorpay	Test	On-ramp (Payment Collection)
RazorpayX	Test	Off-ramp (Payouts)

---

5. PROJECT STRUCTURE

```
nivix-project/
├── 📁 bridge-service/                # Node.js API Gateway
│   ├── src/
│   │   ├── index.js                  # Main Express server (75K+ lines)
│   │   ├── onramp/                   # On-ramp modules
│   │   │   ├── onramp-engine.js      # Order → Payment → Token delivery
│   │   │   ├── razorpay-payment-gateway.js
│   │   │   ├── order-manager.js
│   │   │   └── crypto-delivery-service.js
│   │   ├── offramp/                  # Off-ramp modules
│   │   │   └── offramp-engine.js     # Token burn → Fiat payout
│   │   ├── payments/                 # Payment gateways
│   │   │   ├── razorpay-gateway.js   # Off-ramp payouts
│   │   │   ├── razorpayx-payouts.js  # RazorpayX API
│   │   │   └── fiat-payout-service.js
│   │   ├── treasury/                 # Treasury management
│   │   │   └── treasury-manager.js
│   │   ├── stablecoin/               # Exchange rates
│   │   │   └── exchange-rate-service.js
│   │   ├── solana/                   # Blockchain clients
│   │   │   ├── solana-client.js
│   │   │   └── anchor-client.js
│   │   └── compliance/               # KYC integration
│   │       └── kyc-service.js
│   ├── .env                          # Environment configuration
```

```
│   └── package.json
│
├── 📁 frontend/                        # React Frontend
│   └── nivix-pay-old/
│       ├── src/
│       │   ├── pages/
│       │   │   ├── Dashboard.tsx
│       │   │   ├── Send.tsx
│       │   │   ├── KYC.tsx
│       │   │   ├── KYCAdmin.tsx
│       │   │   ├── PaymentApp.tsx
│       │   │   └── OfframpTesting.tsx
│       │   ├── components/
│       │   └── services/
│       │       └── apiService.ts
│       └── package.json
│
├── 📁 solana/                          # Solana Smart Contracts
│   └── nivix_protocol/
│       ├── programs/nivix_protocol/
│       │   └── src/lib.rs             # Main Anchor program
│       ├── tests/
│       ├── Anchor.toml
│       └── Cargo.toml
│
├── 📁 fabric-samples/                  # Hyperledger Fabric Network
│   └── test-network/
│       └── chaincode-nivix-kyc/       # KYC chaincode
│
├── 📁 REPORTS/                         # Documentation
│   ├── NIVIX_TECHNICAL_DOCUMENTATION.md
│   ├── OFFRAMP_REPORT.md
│   └── NIVIX_FRONTEND_SPECIFICATIONS.md
│
├── 📄 start-nivix.sh                   # Start all services
├── 📄 stop-nivix.sh                    # Stop all services
└── 📄 WALLETS_REGISTRY.json            # Wallet configuration
```

---

6. BLOCKCHAIN INTEGRATION

6.1 Solana Smart Contract

Program ID (Devnet): `FavSaLCcw6qgpLob47uGPoNhJRsGjBMB1tSb7CZTavbw`

Main Functions

```rust
// Initialize platform
pub fn initialize_platform(ctx: Context<InitializePlatform>) -> Result<()>

// Create liquidity pool
pub fn create_liquidity_pool(
    ctx: Context<CreateLiquidityPool>,
    currency_a: String,
    currency_b: String,
    initial_liquidity_a: u64,
```

```
    initial_liquidity_b: u64,
) -> Result<()>

// Process transfer
pub fn process_transfer(
    ctx: Context<ProcessTransfer>,
    amount: u64,
    source_currency: String,
    destination_currency: String,
    recipient_wallet_seed: [u8; 32],
    memo: String,
) -> Result<()>

// Swap currencies
pub fn swap_currencies(
    ctx: Context<SwapCurrencies>,
    amount_in: u64,
    minimum_amount_out: u64,
) -> Result<()>
```

Account Structures

```
pub struct Platform {
    pub admin: Pubkey,
    pub fee_rate: u64,
    pub total_transactions: u64,
    pub supported_currencies: Vec<String>,
    pub bump: u8,
}

pub struct User {
    pub wallet_address: Pubkey,
    pub kyc_verified: bool,
    pub total_sent: u64,
    pub total_received: u64,
    pub risk_score: u8,
    pub bump: u8,
}

pub struct LiquidityPool {
    pub name: String,
    pub admin: Pubkey,
    pub source_currency: String,
    pub destination_currency: String,
    pub exchange_rate: u64,        // Scaled by 10,000
    pub pool_fee_rate: u64,        // Basis points
    pub total_swapped: u64,
    pub is_active: bool,
}
```

6.2 Hyperledger Fabric Chaincode

Chaincode Name: `nivix-kyc`
Channel: `mychannel`

KYC Functions

```
// Store KYC data
async StoreKYC(ctx, userId, solanaAddress, fullName, kycVerified, verificationDate, riskScore,
countryCode)

// Get KYC status
async GetKYCStatus(ctx, solanaAddress)

// Update KYC risk score
async UpdateKYCRisk(ctx, solanaAddress, newRiskScore)

// Validate transaction
async ValidateTransaction(ctx, transactionData)

// Query all KYC records
async QueryAllKYC(ctx)
```

KYC Data Structure

```
{
    "userId": "user_123",
    "solanaAddress": "8VyJ4EgKeto2vhVzq2wgwD9GrFz1wcWnGHP97buwxZj2",
    "fullName": "John Doe",
    "kycVerified": true,
    "verificationDate": "2025-09-08T05:00:00Z",
    "riskScore": 3,
    "countryCode": "USA"
}
```

---

7. PAYMENT GATEWAY INTEGRATION

7.1 Razorpay (On-Ramp)

Purpose: Collect fiat payments from users

File: `bridge-service/src/onramp/razorpay-payment-gateway.js`

Configuration

```
# .env file
RAZORPAY_KEY_ID=rzp_test_ReyC3sUcY6cCtH
RAZORPAY_KEY_SECRET=Wk003BQlVsiX594MM1lYeJoa
```

Key Features

- ✅ Payment order creation
- ✅ Signature verification (HMAC SHA256)
- ✅ Webhook handling
- ✅ Multiple payment methods (UPI, Cards, Net Banking)

API Usage

```
const RazorpayPaymentGateway = require('./razorpay-payment-gateway');
const gateway = new RazorpayPaymentGateway();

// Create payment order
const order = await gateway.createPaymentOrder({
    amount: 10000,        // ₹100 in paise
    currency: 'INR',
    userAddress: 'solana_wallet_address',
    cryptoAmount: 1.2,
    cryptoToken: 'USD',
    orderId: 'order_123'
});

// Verify payment signature
const isValid = gateway.verifyPaymentSignature(paymentId, orderId, signature);
```

7.2 RazorpayX (Off-Ramp)

Purpose: Send fiat payouts to user bank accounts

File: `bridge-service/src/payments/razorpayx-payouts.js`

Configuration

```
# .env file
RAZORPAY_KEY_ID=rzp_test_ReyC3sUcY6cCtH
RAZORPAY_KEY_SECRET=Wk003BQlVsiX594MM1lYeJoa
RAZORPAY_ACCOUNT_NUMBER=2323230040290482
```

Payout Modes

Mode	Amount Limit	Processing Time	Availability
UPI	Up to ₹5 lakh	Instant (2 min)	24x7
IMPS	Up to ₹5 lakh	Instant (2 min)	24x7
NEFT	No limit	2 hours	Business hours
RTGS	Above ₹2 lakh	30 minutes	Business hours

API Usage

```
const RazorpayXPayouts = require('./razorpayx-payouts');
const razorpayx = new RazorpayXPayouts();

// Complete payout flow
const result = await razorpayx.processCompletePayout(
    {
        name: 'John Doe',
        email: 'john@example.com',
        phone: '9876543210',
        bank_account: {
            account_number: '1234567890',
            ifsc_code: 'SBIN0000001',
            account_holder_name: 'John Doe'
```

```
        }
    },
    1000,           // Amount in INR
    'payout',       // Purpose
    'nivix_txn_123' // Reference ID
);
```

3-Step Workflow

1. **Create Contact** → Register beneficiary
2. **Create Fund Account** → Link bank/UPI to contact
3. **Create Payout** → Transfer money

---

8. KYC/COMPLIANCE SYSTEM

8.1 Architecture

```
User → Frontend → Bridge Service → Hyperledger Fabric
                                          │
                                          ▼
                                    Private Data
                                    Collections
```

8.2 KYC Flow

1. **User Submission** → 4-step KYC form (Personal → Address → Documents → Submit)
2. **Data Storage** → Hyperledger Fabric private collections
3. **Verification** → Admin approval workflow
4. **Status Check** → Real-time KYC status API

8.3 KYC API Endpoints

```
# Submit KYC
POST /api/kyc/submit
{
    "solanaAddress": "...",
    "fullName": "John Doe",
    "email": "john@example.com",
    "phone": "+1234567890",
    "dateOfBirth": "1990-01-01",
    "nationality": "USA",
    "documentType": "passport",
    "documentNumber": "TEST123456"
}

# Check KYC Status
GET /api/kyc/status/:solanaAddress

# Response
{
    "kycVerified": true,
    "riskScore": 3,
    "countryCode": "USA"
}
```

8.4 Compliance Features

- ✅ Private data collections for PII
- ✅ Risk scoring (1-5 scale)
- ✅ Country-based transaction limits
- ✅ Immutable audit trail
- ✅ Multi-organization endorsement

---

9. ON-RAMP SYSTEM

9.1 Flow Diagram

```
User Wallet → Frontend → Bridge Service → Razorpay → Payment Confirmation
                                ↓                              ↓
                         Order Creation              Webhook Handler
                                ↓                              ↓
                       Exchange Rate Calc          Signature Verify
                                ↓                              ↓
                        Feasibility Check           Token Minting
                                ↓                              ↓
                      Razorpay Order ID        Solana Transaction
                                                               ↓
                                                 Confirmation & Notify
```

9.2 Step-by-Step Process

Step 1: Order Creation

- User specifies fiat amount and target crypto
- Real-time exchange rate calculation
- Order validation and feasibility check
- Unique order ID generation

Step 2: Payment Processing

- Razorpay payment order creation
- Secure payment gateway redirect
- Multiple payment methods (UPI, Cards, Net Banking)
- Payment confirmation via webhook

Step 3: Token Delivery

- Verify payment signature (HMAC SHA256)
- Check/Create Associated Token Account (ATA)
- Mint tokens with bridge wallet authority
- Confirm transaction on Solana devnet

Step 4: Completion

- Update order status
- Record transaction on Hyperledger Fabric
- Notify user via frontend
- Generate receipt and transaction ID

9.3 API Endpoints

```
# Create Order
POST /api/onramp/create-order
```

```
{
    "userAddress": "6G74ELnKsDCWDg7w6QCWsZ99dBweiHg55NtAwmpB2KW5",
    "fiatAmount": 1000,
    "fiatCurrency": "INR",
    "cryptoCurrency": "USD"
}

# Create Payment
POST /api/onramp/create-payment
{
    "orderId": "onramp_1757636197763_swsbcv"
}

# Verify Payment (after Razorpay checkout)
POST /api/onramp/verify-payment
{
    "razorpay_order_id": "order_xxx",
    "razorpay_payment_id": "pay_xxx",
    "razorpay_signature": "xxx",
    "orderId": "onramp_xxx"
}

# Check Order Status
GET /api/onramp/order-status/:orderId

# Get User Orders
GET /api/onramp/user-orders/:userAddress
```

9.4 Verified Transactions

```
Transaction Signatures (Solana Devnet):
• 3u9jQ4HiKqdjiVn6hsRmzEYVBFGXUb57TH5tuTGPBBgcdYd4Kcj2RxrkfeXW4Sox2FKECrcrthALPgWioedQ1DKS
• ALbVW5SbakpheGvqeRH6N18GyteRAbL6c3u93uFQ1Fy56SsviovTRakSXpheY7JfALXkmvUfahq2oNQHwMi9egp
• 2uzHQNL8QdGr65FQosDEYBzWTPv54VMRiwuSWZ8QbsQ2BR3xJSFVaYRhk7U3xq9E3e6LjgFTK1F7NbCRhMFbehD6
```

---

10. OFF-RAMP SYSTEM

10.1 Dual Routing Strategy

Route 1: Local Treasury (Fast, Cheap)

```
User Token → Burn Tokens → Treasury Debit → Domestic Bank Transfer
   (Wallet)    (Solana)     (Database)        (UPI/IMPS/NEFT)
```

Route 2: Partner/PSP (Cross-border)

```
User Token → Swap to USDC → Transfer to PSP → FX Conversion → Bank Payout
   (Wallet)   (AMM Pool)      (Solana)         (PSP Desk)     (Local Rails)
```

10.2 Automated Routing Logic

```
function selectTreasuryRoute(fromCountry, toCountry, amount) {
    // Check treasury balance
    if (treasuryBalance - withdrawalAmount >= minimumThreshold) {
```

```
        return "direct";  // Fast local payout
    } else {
        return "hybrid";  // Stablecoin route
    }
}
```

Routing Rules:

- India (INR) → Default: Direct (UPI/IMPS)
- USA (USD) → Default: Hybrid (ACH via partner)
- Europe (EUR) → Default: Hybrid (SEPA via partner)

10.3 API Endpoints

```
# Get Quote
POST /api/offramp/quote
{
    "amount": 100,
    "fromCurrency": "USD",
    "toCurrency": "INR",
    "corridor": "US-IN",
    "userAddress": "..."
}

# Initiate Off-Ramp
POST /api/offramp/initiate
{
    "quoteId": "quote_123456789",
    "userAddress": "...",
    "bankDetails": {
        "accountNumber": "1234567890",
        "ifscCode": "SBIN0000001",
        "accountName": "Test User"
    }
}

# Check Status
GET /api/offramp/status/:transactionId
```

10.4 Fee Structure

Fee Type	Amount	Description
Platform Fee	0.5%	Nivix platform fee
Pool Fee	0.15-0.35%	Liquidity pool swap fee
UPI Fee	₹0-5	UPI transaction fee
IMPS Fee	₹2-5	IMPS transaction fee
NEFT Fee	₹2-5	NEFT transaction fee

---

11. LIQUIDITY POOLS

11.1 Deployed Pools

Pool ID	Currency A	Currency B	Exchange Rate	Fee Rate
Pool-1	USDC	INR	83.25	0.30%
Pool-2	USDC	EUR	0.92	0.25%
Pool-3	USDC	GBP	0.79	0.25%
Pool-4	USDC	JPY	149.50	0.30%
Pool-5	USDC	CAD	1.36	0.25%
Pool-6	USDC	AUD	1.53	0.25%
Pool-7	INR	EUR	0.011	0.35%

11.2 Pool Configuration

```
pub struct LiquidityPool {
    pub name: String,                    // "USD-INR Pool"
    pub admin: Pubkey,                   // Pool administrator
    pub source_currency: String,         // "USD"
    pub destination_currency: String,    // "INR"
    pub source_mint: Pubkey,            // Source token mint
    pub destination_mint: Pubkey,       // Destination token mint
    pub exchange_rate: u64,             // 832500 (83.25 * 10,000)
    pub pool_fee_rate: u64,             // 30 (0.30%)
    pub total_swapped: u64,             // Total volume
    pub is_active: bool,                // Pool status
}
```

11.3 Fee Calculation Example

```
// For a $100 USD → INR swap in USD-INR pool:
const amountIn = 10000;                          // $100 in smallest units
const platformFee = (amountIn * 50) / 10000;     // 0.5% = $0.50
const poolFee = (amountIn * 30) / 10000;         // 0.3% = $0.30
const totalFee = platformFee + poolFee;          // Total = $0.80

// User receives: $100 - $0.80 = $99.20 worth of INR
// At 83.25 rate: 99.20 * 83.25 = ₹8,258.40
```

11.4 API Endpoints

```
# List All Pools
GET /api/pools

# Get Pool Info
GET /api/pools/:poolId

# Swap Currencies
POST /api/pools/swap
{
    "poolId": "pool_1",
```

```
    "amountIn": 1000,
    "minAmountOut": 82000
}
```

---

12. TREASURY MANAGEMENT

12.1 Multi-Tier Treasury Architecture

Tier 1: Direct Local Fiat Providers (Fastest)

Country	Provider	Status
🇮🇳 India	Cashfree/Razorpay	✅ Implemented
🇺🇸 USA	Stripe	⚠️ Pending
🇪🇺 EU	SEPA	⚠️ Pending
🇬🇧 UK	Faster Payments	⚠️ Pending

Tier 2: Stablecoin Bridge (Medium)

```
Local Fiat → USDC/USDT → Local Treasury → Target Fiat
```

Tier 3: Cross-Border Settlement (Slowest)

```
Local Fiat → USDC → International Treasury → Target Fiat
```

12.2 Corridor Configuration

```
{
    "india": {
        "route": "direct",
        "fallbackRoute": "hybrid",
        "paymentMethods": ["UPI", "IMPS", "NEFT"],
        "processingTime": "instant-30min",
        "thresholds": {
            "min": 50000,
            "target": 500000
        }
    }
}
```

12.3 Auto-Rebalancing

```
// Treasury monitoring
if (treasuryBalance < minimumThreshold) {
    // Trigger rebalancing
    await convertStablecoinToFiat(amount);
    await notifyOperations('Treasury rebalanced');
}
```

---

13. API DOCUMENTATION

13.1 Base URLs

Service	URL
Bridge Service	`http://localhost:3002`
Frontend	`http://localhost:3000`
Health Check	`http://localhost:3002/health`

13.2 Health & Status

```
# Health Check
GET /health
Response: {
    "status": "ok",
    "timestamp": "2026-01-01T17:00:00Z",
    "services": {
        "hyperledger": "connected",
        "solana": "connected"
    }
}
```

13.3 KYC Endpoints

Method	Endpoint	Description
POST	`/api/kyc/submit`	Submit KYC data
GET	`/api/kyc/status/:address`	Get KYC status
POST	`/api/kyc/update`	Update verification

13.4 Solana Endpoints

Method	Endpoint	Description
GET	`/api/solana/balance/:address`	Get SOL balance
POST	`/api/solana/airdrop`	Request SOL airdrop (devnet)
POST	`/api/solana/create-mint`	Create token mint
POST	`/api/solana/mint-to`	Mint tokens
POST	`/api/solana/get-or-create-ata`	Get/Create ATA
POST	`/api/solana/build-transfer`	Build transfer TX
POST	`/api/solana/submit-signed`	Submit signed TX
GET	`/api/solana/bridge-wallet`	Get bridge wallet
GET	`/api/solana/token-balance/:address/:mint`	Get token balance
POST	`/api/solana/transfer`	Transfer tokens

13.5 On-Ramp Endpoints

Method	Endpoint	Description
POST	`/api/onramp/create-order`	Create on-ramp order
POST	`/api/onramp/create-payment`	Create Razorpay payment
POST	`/api/onramp/verify-payment`	Verify payment signature
POST	`/api/onramp/razorpay-webhook`	Handle payment webhook
GET	`/api/onramp/order-status/:orderId`	Get order status
GET	`/api/onramp/user-orders/:address`	Get user orders

13.6 Off-Ramp Endpoints

Method	Endpoint	Description
POST	`/api/offramp/quote`	Get withdrawal quote
POST	`/api/offramp/initiate`	Initiate withdrawal
GET	`/api/offramp/status/:txId`	Get transaction status

13.7 Bridge Endpoints

Method	Endpoint	Description
POST	`/api/bridge/initiate-transfer`	Initiate cross-border transfer
GET	`/api/bridge/transaction-status/:id`	Get transaction status
GET	`/api/bridge/wallet-transactions/:address`	Get wallet transactions
POST	`/api/bridge/sync-offline-transaction`	Sync offline TX

13.8 Pool Endpoints

Method	Endpoint	Description
GET	`/api/pools`	List all pools
GET	`/api/pools/:id`	Get pool info
POST	`/api/pools/swap`	Swap currencies

---

14. DEPLOYMENT GUIDE

14.1 Prerequisites

- **Node.js**: >= 18.x
- **Rust**: Latest stable version
- **Solana CLI**: v1.16.0+
- **Anchor CLI**: Latest version
- **Docker**: For Hyperledger Fabric
- **Git**: Version control

14.2 Quick Start

```
# Navigate to project
cd nivix-project

# Start all services
./start-nivix.sh

# Stop all services
./stop-nivix.sh
```

14.3 Manual Start (Step by Step)

1. Start Hyperledger Fabric

```
cd fabric-samples/test-network
./network.sh down
./network.sh up createChannel -ca -c mychannel
./network.sh deployCC -ccn nivix-kyc -ccp ./chaincode-nivix-kyc -ccl go -c mychannel
```

2. Start Bridge Service

```
cd bridge-service

# Set environment variables
export NODE_ENV=development
export RAZORPAY_KEY_ID="rzp_test_ReyC3sUcY6cCtH"
export RAZORPAY_KEY_SECRET="Wk003BQlVsiX594MM1lYeJoa"
export RAZORPAY_ACCOUNT_NUMBER="2323230040290482"

npm start
```

3. Start Frontend

```
cd frontend/nivix-pay-old
npm start
```

14.4 Environment Configuration

```
# bridge-service/.env

# Environment
NODE_ENV=staging
PORT=3002

# Solana Configuration (Devnet)
SOLANA_NETWORK=devnet
SOLANA_RPC_URL=https://api.devnet.solana.com

# Razorpay Configuration (Test Mode)
RAZORPAY_ENV=test
RAZORPAY_KEY_ID=rzp_test_ReyC3sUcY6cCtH
```

```
RAZORPAY_KEY_SECRET=Wk003BQlVsiX594MM1lYeJoa
RAZORPAY_ACCOUNT_NUMBER=2323230040290482

# Feature Flags
ENABLE_OFFRAMP=true
ENABLE_TREASURY_MANAGEMENT=true
ENABLE_STABLECOIN_BRIDGE=true
```

14.5 Production Deployment

Server Requirements

- Ubuntu 20.04+ LTS
- 4GB RAM minimum (8GB recommended)
- 50GB SSD storage
- Static IP address
- Domain name configured
- SSL certificate (Let's Encrypt)

Nginx Configuration

```
server {
    listen 443 ssl;
    server_name your-domain.com;

    ssl_certificate /etc/letsencrypt/live/your-domain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/your-domain.com/privkey.pem;

    # Frontend
    location / {
        root /var/www/nivix/frontend/dist;
        try_files $uri $uri/ /index.html;
    }

    # Backend API
    location /api {
        proxy_pass http://localhost:3002;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

---

15. COMMANDS REFERENCE

15.1 Service Management

```
# Start Everything
./start-nivix.sh

# Stop Everything
./stop-nivix.sh
```

```
# Start Bridge Service Only
cd bridge-service && npm start

# Start Frontend Only
cd frontend/nivix-pay-old && npm start
```

15.2 Health Checks

```
# Check Bridge Service
curl http://localhost:3002/health

# Check Pools
curl http://localhost:3002/api/pools

# Check KYC Status
curl http://localhost:3002/api/kyc/status/YOUR_WALLET_ADDRESS
```

15.3 On-Ramp Testing

```
# Create Order
curl -X POST http://localhost:3002/api/onramp/create-order \
  -H "Content-Type: application/json" \
  -d '{
    "userAddress": "6G74ELnKsDCWDg7w6QCWsZ99dBweiHg55NtAwmpB2KW5",
    "fiatAmount": 1000,
    "fiatCurrency": "INR",
    "cryptoCurrency": "USD"
  }'

# Create Payment
curl -X POST http://localhost:3002/api/onramp/create-payment \
  -H "Content-Type: application/json" \
  -d '{"orderId": "YOUR_ORDER_ID"}'
```

15.4 Off-Ramp Testing

```
# Get Quote
curl -X POST http://localhost:3002/api/offramp/quote \
  -H "Content-Type: application/json" \
  -d '{
    "amount": 100,
    "fromCurrency": "USD",
    "toCurrency": "INR",
    "corridor": "US-IN",
    "userAddress": "YOUR_WALLET_ADDRESS"
  }'
```

15.5 Troubleshooting Commands

```
# Kill Process on Port
lsof -ti:3002 | xargs kill -9  # Bridge Service
lsof -ti:3000 | xargs kill -9  # Frontend
```

```
# Check Running Processes
ps aux | grep node
lsof -i :3002

# View Logs
tail -f bridge-service/logs/bridge.log
```

15.6 Hyperledger Fabric Commands

```
# Start Network
cd fabric-samples/test-network
./network.sh up createChannel -ca -c mychannel

# Deploy Chaincode
./network.sh deployCC -ccn nivix-kyc -ccp ./chaincode-nivix-kyc -ccl go

# Stop Network
./network.sh down

# Test Chaincode
./fabric-invoke.sh "QueryAllKYC" "[]" "query"
```

---

16. SECURITY & KEY MANAGEMENT

16.1 Key Types

Key Type	Storage	Rotation
Razorpay API Keys	Environment Variables	90 days
Razorpay Webhook Secret	Environment Variables	180 days
Solana Treasury Keypair	Encrypted file	Never (backup secured)
Database Credentials	Environment Variables	90 days
JWT Secret	Environment Variables	90 days

16.2 Security Measures

Technical Security

- ✅ AES-256 encryption for data at rest
- ✅ TLS/SSL for all communications
- ✅ Role-based access control (RBAC)
- ✅ Comprehensive audit logging
- ✅ Payment signature verification (HMAC SHA256)

Operational Security

- ✅ Never commit credentials to git
- ✅ Use environment variables only
- ✅ Regular key rotation schedule
- ✅ Incident response plan documented

16.3 Environment Variables Security

```
# NEVER commit .env to git
# Add to .gitignore:
.env
.env.production
*.pem
*.key
```

16.4 Key Rotation Script

```bash
#!/bin/bash
# rotate-keys.sh

echo "🔄 Starting key rotation..."

# Generate new secrets
NEW_SECRET=$(openssl rand -base64 32)

# Update secrets manager (example)
# aws secretsmanager update-secret --secret-id "nivix/production/key" --secret-string "$NEW_SECRET"

# Restart services
pm2 restart nivix-backend

# Verify health
curl -f http://localhost:3002/health || exit 1

echo "✅ Key rotation complete!"
```

---

17. LEGAL & REGULATORY COMPLIANCE

17.1 India Cryptocurrency Regulations (2025)

Legal Status

- **Cryptocurrency**: Legal to hold, buy, sell, and trade
- **Classification**: Virtual Digital Assets (VDAs) under Income Tax Act
- **Mining**: Not explicitly illegal, no licensing required

Key Regulatory Bodies

- **FIU-IND**: Registration mandatory for VDASPs under PMLA
- **RBI**: Regulates banks interacting with crypto businesses
- **SEBI**: Monitoring for potential securities classification
- **Ministry of Finance**: Tax policy

17.2 Compliance Requirements

FIU-IND Registration (CRITICAL)

- **Status**: Required before production launch
- **Obligations**:
  - ✅ Full KYC for all users
  - ✅ Record maintenance (5 years)
  - ✅ Suspicious Transaction Reports (STR)
  - ✅ High-value transaction reporting

Tax Framework

Tax Type	Rate	Application
VDA Income Tax	30% + 4% cess	Profits from crypto
TDS	1%	Transactions > ₹50,000
GST	18%	Platform services

Data Protection (DPDPA 2023)

- ✅ Explicit user consent required
- ✅ Purpose limitation on data use
- ✅ Data minimization
- ✅ Security safeguards
- ✅ Breach notification
- ✅ User rights (access, correction, erasure)

17.3 Nivix Compliance Status

Requirement	Status	Notes
KYC System	✅ Implemented	Hyperledger Fabric
Transaction Monitoring	✅ Active	All transactions logged
FIU-IND Registration	⚠️ Pending	Before production
TDS Implementation	✅ Ready	Infrastructure in place
DPDPA Compliance	✅ Implemented	Privacy policy, consent
Banking Relationship	⚠️ Pending	Required for production

17.4 Pre-Production Legal Checklist

- ☐ Company incorporation (NivixPe Private Limited)
- ☐ GST registration
- ☐ FIU-IND registration (PMLA)
- ☐ RBI payment aggregator authorization (if required)
- ☐ Bank account opening
- ☐ Tax registrations (TAN, PAN)
- ☐ Terms of Service
- ☐ Privacy Policy (DPDPA compliant)
- ☐ KYC/AML Policy

---

18. PRODUCTION READINESS CHECKLIST

18.1 Security & Keys

- ☐ All production keys generated and secured
- ☐ Payment gateway production accounts configured
- ☐ SSL certificates installed and valid
- ☐ Firewall rules configured
- ☐ Environment variables properly set

- [ ] Key rotation schedule established
- [ ] Backup and recovery procedures tested

18.2 Infrastructure

- [ ] Production server provisioned (Ubuntu 20.04+)
- [ ] Domain name configured and pointing to server
- [ ] Load balancer configured (if needed)
- [ ] Database cluster configured
- [ ] Monitoring tools installed

18.3 Payment Gateways

- [ ] Razorpay production account activated
- [ ] RazorpayX production account activated
- [ ] Webhook endpoints configured
- [ ] Test transactions completed successfully

18.4 Blockchain Integration

- [ ] Solana mainnet connection tested
- [ ] Treasury wallet funded and secured
- [ ] Hyperledger Fabric production network deployed
- [ ] KYC chaincode deployed and tested

18.5 Testing

- [ ] Unit tests passing
- [ ] Integration tests passing
- [ ] End-to-end tests passing
- [ ] Load testing completed
- [ ] Security testing completed
- [ ] Payment gateway testing
- [ ] Blockchain integration testing

18.6 Go-Live

- [ ] DNS propagation complete
- [ ] SSL certificates active
- [ ] All services accessible
- [ ] Payment gateways active
- [ ] Monitoring alerts configured
- [ ] Team ready for support

---

19. TROUBLESHOOTING

19.1 Common Issues

Port Already in Use

```
# Solution
lsof -ti:3002 | xargs kill -9
```

```
lsof -ti:3000 | xargs kill -9
```

Service Not Starting

```
# Check environment variables
echo $NODE_ENV
echo $RAZORPAY_KEY_ID

# Check logs
tail -f bridge-service/logs/bridge.log
```

Frontend Not Loading

```
# Clear cache and restart
cd frontend/nivix-pay-old
rm -rf node_modules package-lock.json
npm install
npm start
```

Blockchain Connection Failed

```
# Verify Solana RPC
curl https://api.devnet.solana.com \
  -X POST \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","id":1,"method":"getHealth"}'
```

19.2 RazorpayX Issues

Zero Deduction Problem

1. Verify `RAZORPAY_ACCOUNT_NUMBER` is set in `.env`
2. Check account balance in RazorpayX Dashboard
3. Ensure test mode credentials are correct

```
# Check configuration
echo $RAZORPAY_ACCOUNT_NUMBER
```

Payout Failures

1. Check error message in logs
2. Verify beneficiary details
3. Check account balance
4. Review RazorpayX Dashboard

19.3 Hyperledger Fabric Issues

Chaincode Deployment Failed

```
cd fabric-samples/test-network
./network.sh down
docker system prune -f
```

```
./network.sh up createChannel -ca -c mychannel
./network.sh deployCC -ccn nivix-kyc -ccp ./chaincode-nivix-kyc -ccl go
```

Connection Failed

```
# Verify Docker containers
docker ps | grep hyperledger

# Check peer logs
docker logs peer0.org1.example.com
```

---

20. PROJECT STATUS & ROADMAP

20.1 Current Completion

Component	Status	%
Core Infrastructure	✅ Complete	100%
Solana Smart Contracts	🔄 In Progress	60%
Hyperledger Fabric	🔄 In Progress	80%
Bridge Service	🔄 In Progress	70%
Frontend Application	⚠️ Needs Work	35%
Payment Processing	⚠️ Needs Work	40%
Treasury System	❌ Early Stage	25%
Compliance & KYC	🔄 In Progress	60%

Overall Project Completion: ~55%

20.2 Milestones Completed

- ✅ **Milestone 1**: Platform Architecture & Setup
- ✅ **Milestone 2**: Core Smart Contracts
- ✅ **Milestone 3**: Compliance Layer (Hyperledger Fabric)
- ✅ **Milestone 4**: Bridge Service & API Layer
- ✅ **Milestone 5**: Payment Gateway Integration (Razorpay)
- ✅ **Milestone 6**: Frontend Development
- 🔄 **Milestone 7**: Multi-Currency Liquidity Pools (In Progress)
- ⏳ **Milestone 8**: Off-Ramp System (Ready for Testing)

20.3 Roadmap

Phase 1: Core Completion (2-3 weeks)

- ☐ USA Stripe Integration
- ☐ Real-time Exchange Rates
- ☐ Multi-tier Routing Logic

Phase 2: Advanced Features (2-3 weeks)

- ☐ Crypto-Fiat Bridge Services
- ☐ Treasury Management System

- ☐ EU Payment Integration

Phase 3: Production Readiness (2-3 weeks)

- ☐ Cross-Border Features
- ☐ Compliance & Security Audit
- ☐ Production Deployment

20.4 Success Metrics

Metric	Target
Uptime	99.9%
Response Time	< 2 seconds
Payment Success Rate	95%+
Security Vulnerabilities	0 critical

📞 SUPPORT & CONTACTS

Technical Support

- **Repository**: GitHub - Quadwinner/Nivix
- **Bridge Service**: Port 3002
- **Frontend**: Port 3000

Emergency Contacts

- **Razorpay Support**: https://razorpay.com/support/
- **Solana Documentation**: https://docs.solana.com/
- **Hyperledger Fabric Docs**: https://hyperledger-fabric.readthedocs.io/

📄 DOCUMENT HISTORY

Version	Date	Changes
1.0	2025-09-08	Initial documentation
2.0	2026-01-01	Consolidated master documentation

🎉 End of Master Documentation

This document consolidates all Nivix Protocol documentation into a single comprehensive reference.

Status: Development/Testing Phase (Solana Devnet)
Last Updated: January 1, 2026